Optimization strategies

Bayesian Global Optimization in Materials Design Using the Bgolearn Platform

Bin CAO (曹斌)
Supervisor : Prof. ZHANG Tong-Yi (张统一)

Guangzhou Municipal Key Laboratory of Materials Informatics
The Hong Kong University of Science and Technology (Guangzhou)

July 7th, 2025

binjacobcao@gmail.com

Target of Optimization

The optimization algorithms applied to black-box problems are collectively geared towards solving the **challenge of effective sampling.**

$x_1 \rightarrow f(x_1)$
$x_2 \rightarrow f(x_2)$
...
$x_n \rightarrow f(x_n)$
(n is **a finite number**)

$x_* \rightarrow f(x_*)$

Many optimization algorithms leverage historical information to guide the selection of the next sampling point, determining which infill data,

$\boldsymbol{x_{n+1}} \rightarrow \boldsymbol{f}(\boldsymbol{x_{n+1}})$ is most informative in the pursuit of deriving the optimal solution x_*.

Non-convex Optimization

Goal : solve argmin_x f(x)

Where f(x) is a black box and is a time-consuming-to-evaluate function

Heuristic optimization algorithm:

Particle swarm algorithm
Simulated annealing algorithm
Genetic algorithm
…

Bayesian optimization:

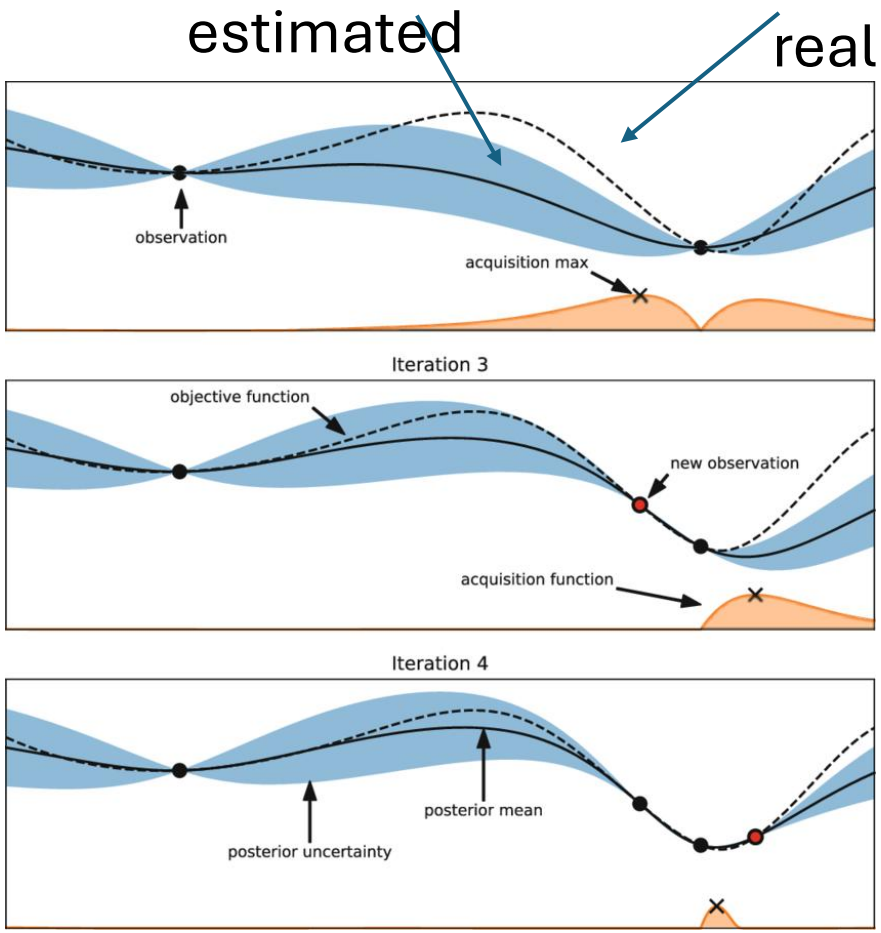
Expected Improvement algorithm
Upper confidence bound
Predictive Entropy Search
Knowledge Gradient
…

Difficulties :

The analytical expression is **not known**
The computation of f(x) is **computationally expensive**

The differences

All optimization algorithms incorporate a fundamental concept of an optimization (response) surface.



[figure from internet]

The optimization surface **undergoes changes** with the introduction of infill data. This response surface can be **implicit**, as seen in heuristic optimization, or explicit, as **observed** in Bayesian optimization.

A notable distinction lies in Bayesian optimization's approach, as **it does not unconditionally trust the derived response surface**. Instead, it carefully navigates the **exploration-exploitation trade-off**.

Baysian global Optimization

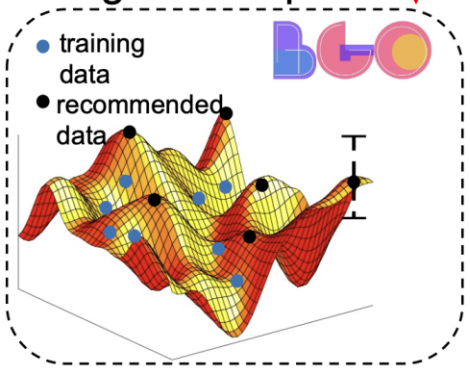
History informations

datasets

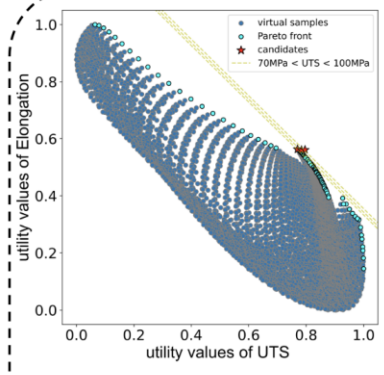
Sn	Ag	Cu	Bi	In	Ti	UTS	EL
Bal.	1	0.5	3	2.5	0.1	64.60	28.85
Bal.	1	0.5	4	2.5	0.1	77.65	16.44
Bal.	1	0.5	5	3.5	0.3	89.85	11.91
⋮							

Response surface

gaussian proc.



optimization



Sn	Ag	Cu	Bi	In	Ti
Bal.	1	0.5	5.5	5.5	0.7
Bal.	1	0.5	2.5	3.3	0.1
Bal.	1	0.5	1.5	4.4	0.2
⋮					11,767

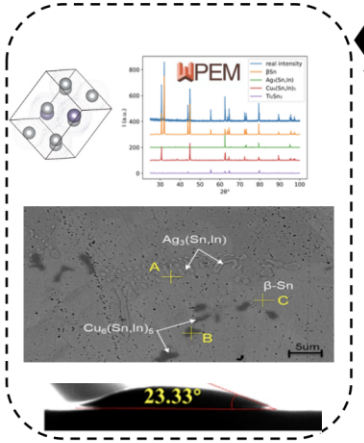
satisfied ?

No

Yes

New infill data

Sn	Ag	Cu	Bi	In	Ti	UTS	EL
Bal.	1	0.5	5.5	5.5	0.7	106.7	11.8
Bal.	1	0.5	2.5	3.3	0.1	62.7	26.4
Bal.	1	0.5	1.5	4.4	0.2	73.9	24.4



characterization

experimental validation

Multi-targets balance
& candidate evaluation

Bgolearn Platform

Vanilla Bgolearn

Interface

Multi-targets

Pareto Balance

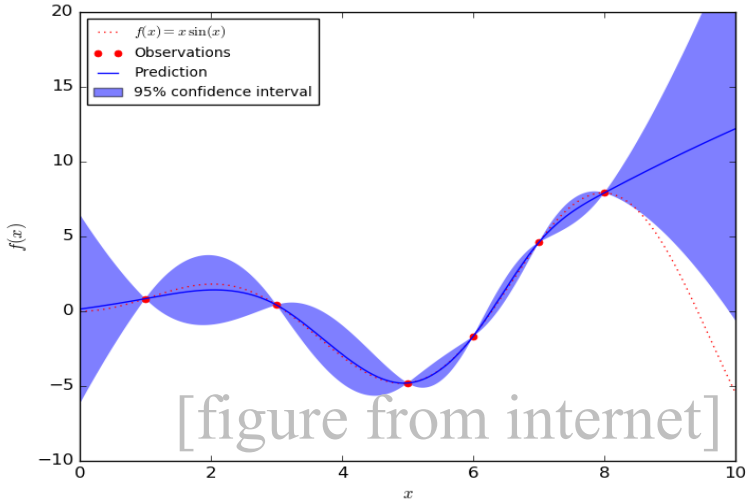
Cao et al. Materials & Design 241 (2024): 112921.

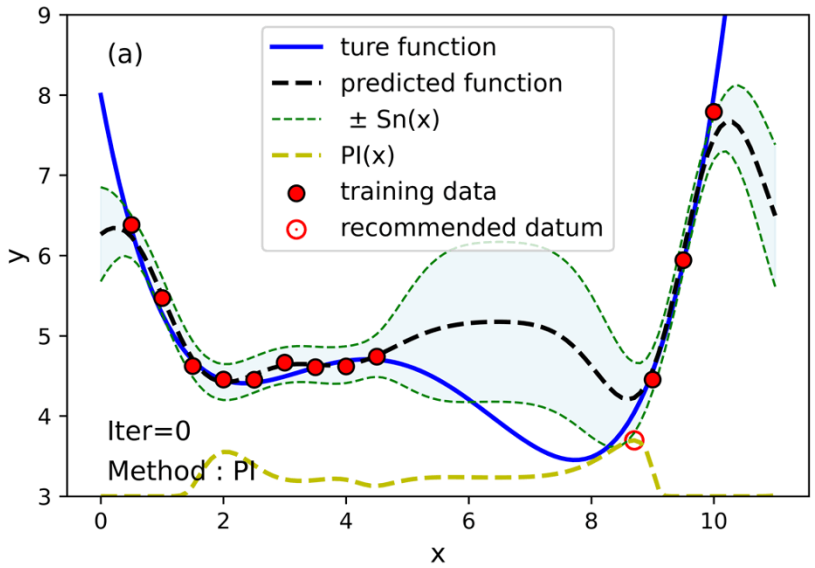
Baysian global Optimization : Response surface

The first and most important step is to **establish the response surface** with collect data pairs

$x_1 \rightarrow f(x_1)$
$x_2 \rightarrow f(x_2)$
...
$x_n \rightarrow f(x_n)$

A ML model





Iterations	x_i	$y_i(x_i)$
1-th	8.7000	4.0418

The reliability is insufficient

Bgolearn provided **three modules** for establishing the surface :

1. Gaussian Process-Based
2. Bootstrip Sampling-Based (SVM, RF, MLP, etc.)
3. Multi-model-Based

Baysian global Optimization : Virtual space

In BGO, the virtual space refers to the area constituted by all **unevaluated data** at the current step.

$x_1 \rightarrow f(x_1)$
$x_2 \rightarrow f(x_2)$
...
$x_n \rightarrow f(x_n)$

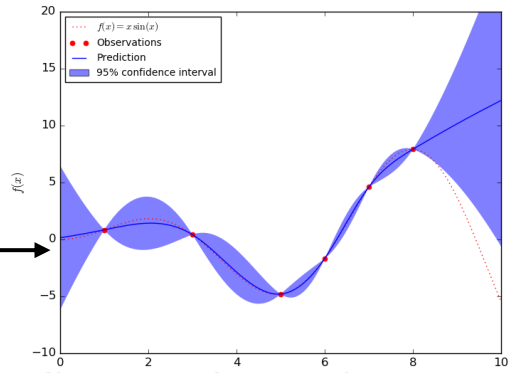
Data be recorded

$V{:}x_i\big(i \notin (1,2,\dots n)\big)$

The data within the virtual space, denoted as V, must encompass the optimal solution

$x_1 \rightarrow f(x_1)$
$x_2 \rightarrow f(x_2)$
...
$x_n \rightarrow f(x_n)$

ML mdoel



[figure from internet]

All data within the virtual space are assessed by the response surface, **taking into account associated uncertainties**

$V{:}x_i\big(i \notin (1,2,\dots n)\big)$

$G{:}g(x_i)\big(i \notin (1,2,\dots n)\big)$

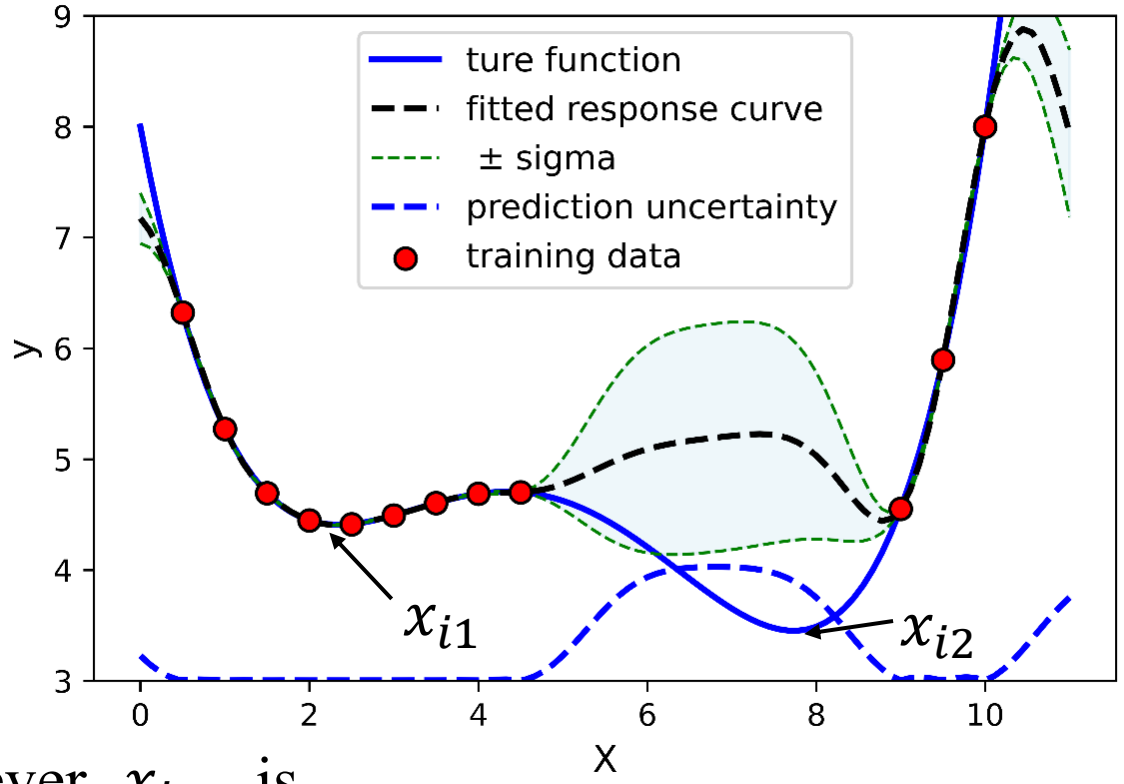
Baysian global Optimization : Ranking

Through the computation of the response surface, we can obtain all the evaluated function values and **their associated uncertainties** for each data point in V

$$V : x_i\big(i \notin (1,2, \dots n)\big)$$

$$G: g(x_i)\big(i \notin (1,2, \dots n)\big)$$
$$S : \mathrm{std}(x_i)\big(i \notin (1,2, \dots n)\big)$$

We use 'g' to denote the evaluated **means** and 'std' to represent the evaluated **standard variance.**



While x_{i1} exhibits the lowest $g(x_{i1})$, however x_{i2} is considered optimal from a global perspective. How can BGO be guided to recommend solutions closer to x_{i2}?

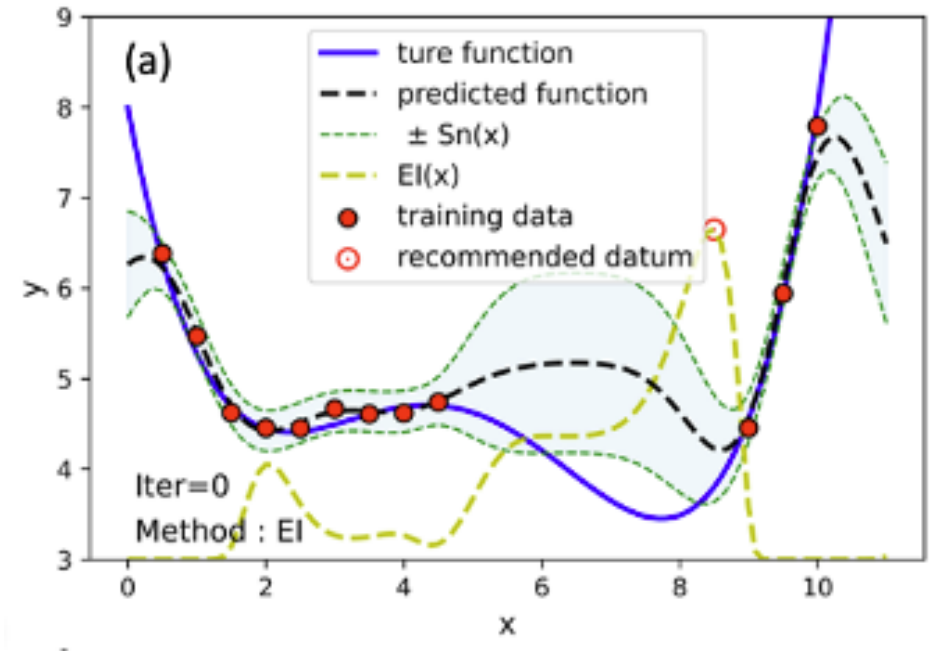
Exploration and Exploitation

The utility function is incorporated into BGO, exemplified by the Expectation Improvement algorithm

$$I_n(\boldsymbol{x}) = max\left(f_{min}^{(n)} - g(\boldsymbol{x}), 0\right)$$

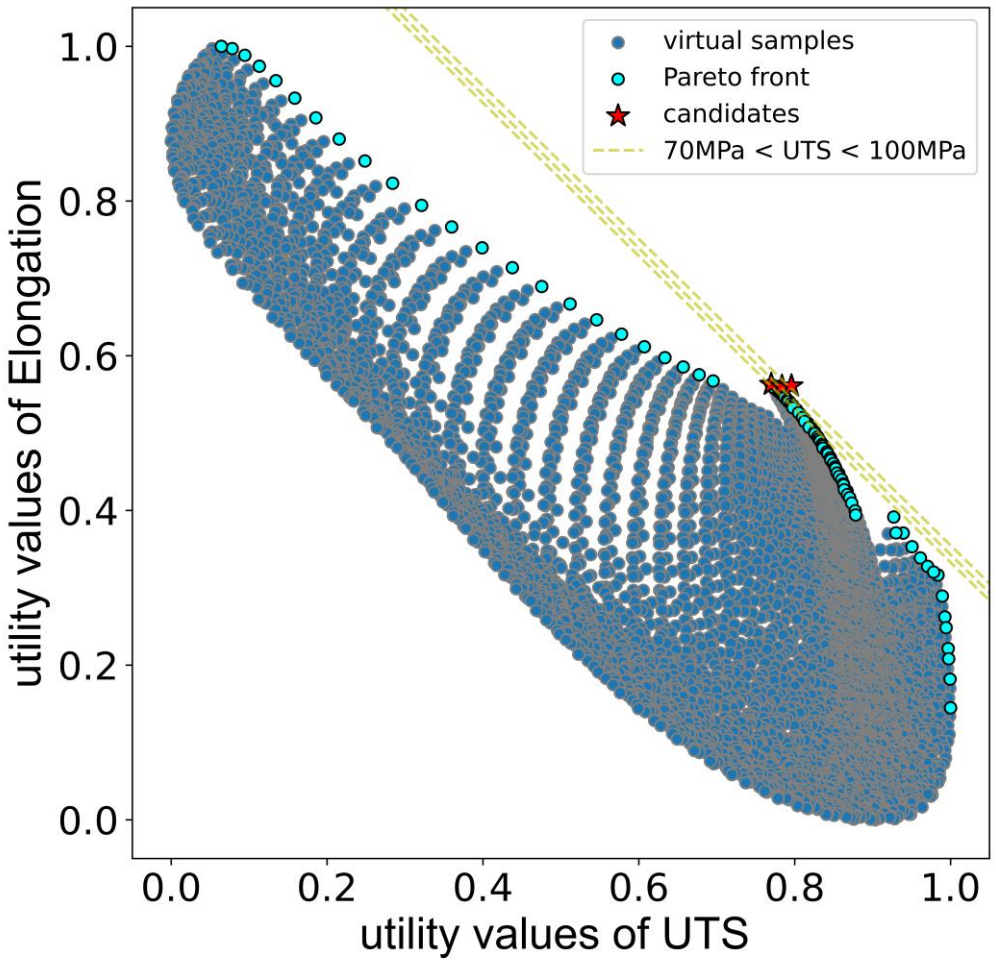
$$EI_n(\boldsymbol{x}) = E_{\hat{y}_n(\boldsymbol{x})}\left(max\left(f_{min}^{(n)} - \hat{g}_n(\boldsymbol{x}), 0\right)\right)$$

$$EI_n(\boldsymbol{x}) = s_n\left[\left(\frac{f_{min}^{(n)} - g_n}{s_n}\right)\Phi\left(\frac{f_{min}^{(n)} - g_n}{s_n}\right) + \phi\left(\frac{f_{min}^{(n)} - g_n}{s_n}\right)\right]$$



Recommendations are made **by considering both the evaluated values and their associated uncertainties**. The recommended data must then undergo measurement to acquire the real function value (experiments or simulations)

Multi-targets selection (**BgoKit**) :



Selecting a data point from the Pareto front involves various methods:

1.Diagonal search

2.Distance search

3.Equal value search

4.Crowding distance search

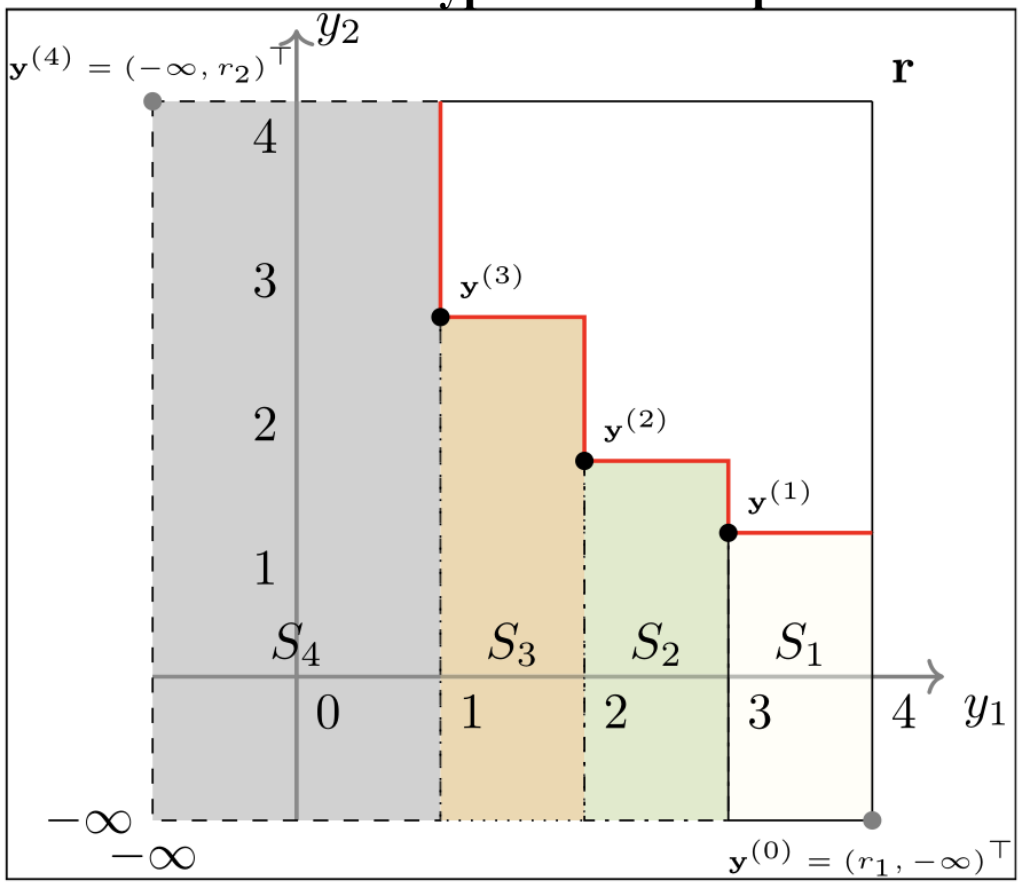
5. …

Each method **recommends points located on the front**, contributing to the multi-task optimization process.

Multi-targets BGO (**MultiBgolearn**) :

In multi-target BGO, the evaluation of infill data focuses on enhancing the influence **across all targets**.

Hypervolume Improvement



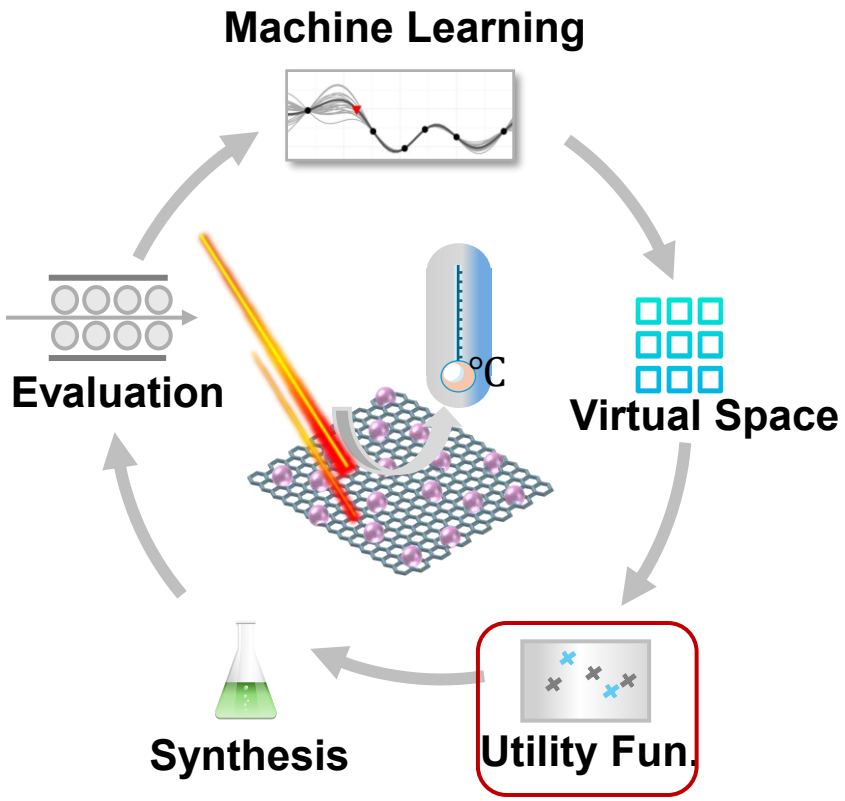
Algorithm 1 MOBGO algorithm.

Input: Objective functions $\mathbf{y}$, initialization size μ, termination criterion T_c
Output: Pareto-front approximation $\mathcal{P}$

Initialize μ points $(\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(\mu)})$;
Evaluate the initial set of μ points: $(\mathbf{y}^{(1)} = \mathbf{y}(\mathbf{x}^{(1)}), \ldots, \mathbf{y}^{(\mu)} = \mathbf{y}(\mathbf{x}^{(\mu)}))$;
Store $(\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(\mu)})$ and $(\mathbf{y}^{(1)} = \mathbf{y}(\mathbf{x}^{(1)}), \ldots, \mathbf{y}^{(\mu)} = \mathbf{y}(\mathbf{x}^{(\mu)}))$ in D:
  $D = ((\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), \ldots, (\mathbf{x}^{(\mu)}, \mathbf{y}^{(\mu)}))$;
Compute the non-dominated subset of D and store it in $\mathcal{P}$;
$g = \mu$;
while $g <= T_c$ **do**
  Train Kriging models $M_1, \cdots, M_d$ based on D ;
  Use an optimizer (*opt*) to find the promising point $\mathbf{x}^*$ based on surrogate models M, with the infill criterion C;
  Update D: $D = D \cup (\mathbf{x}^*, \mathbf{y}(\mathbf{x}^*))$;
  Update $\mathcal{P}$ as the non-dominated subset of D;
  $g = g + 1$;
Return $\mathcal{P}$

[figure from https://doi.org/10.1016/j.swevo.2018.10.007]

Application Cases :



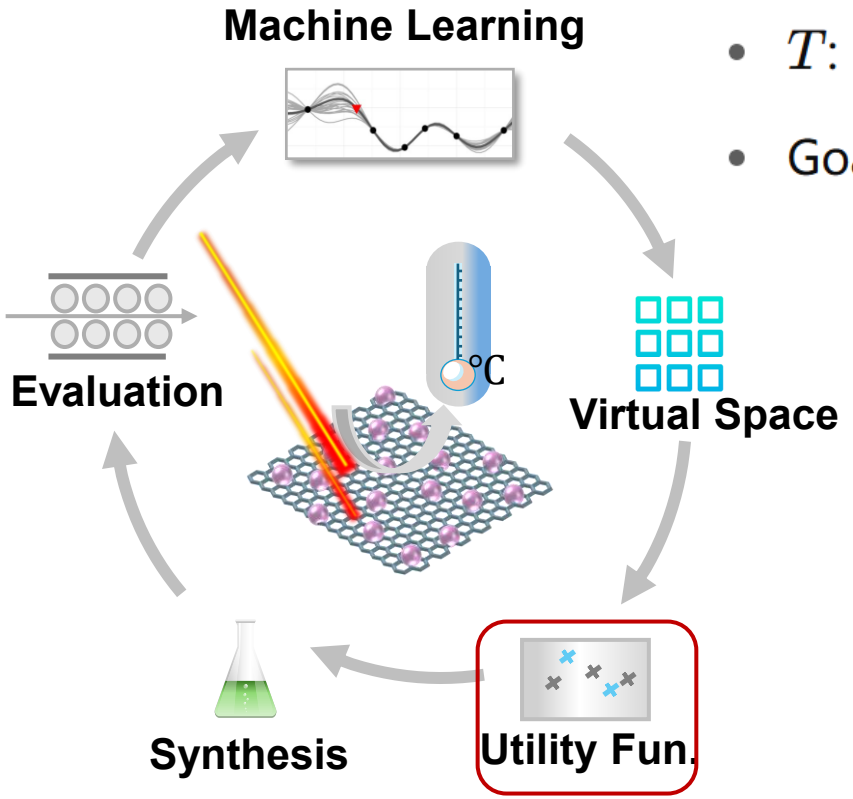
We use the Upper Confidence Bound (UCB) strategy to guide decision-making in our optimization.

UCB balances exploration (uncertainty) and exploitation (expected performance).

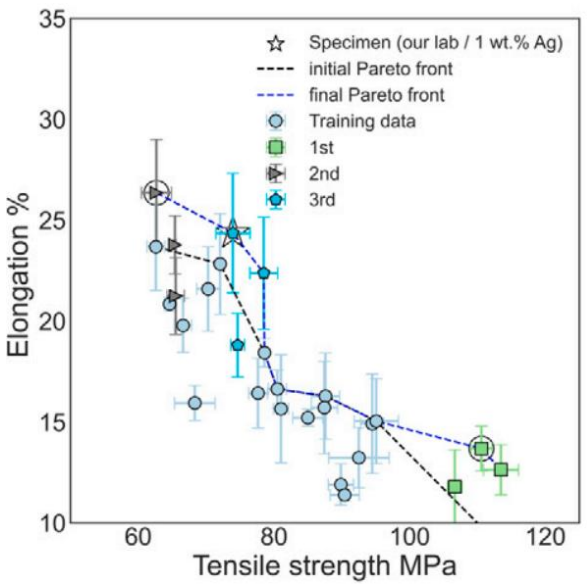
- The UCBs are defined as:
  - $y_{i1}(x_i) = \mu_{i1}(x_i) + \tau\sigma_{i1}(x_i)$
  - $y_{i2}(x_i) = \mu_{i2}(x_i) + \tau\sigma_{i2}(x_i)$
- Where:
  - μ: Predicted **mean** from GPR
  - σ: Predicted **standard deviation** (uncertainty)
  - τ: Weighting parameter controlling exploration vs. exploitation

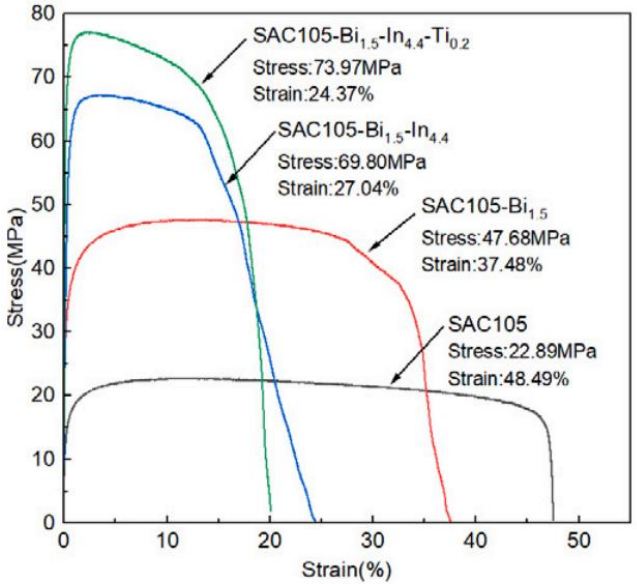
Cao, B., Su, T., Yu, S., Li, T., Zhang, T., Zhang, J., ... & Zhang, T. Y. (2024). Materials & Design, 241, 112921.

Application Cases :



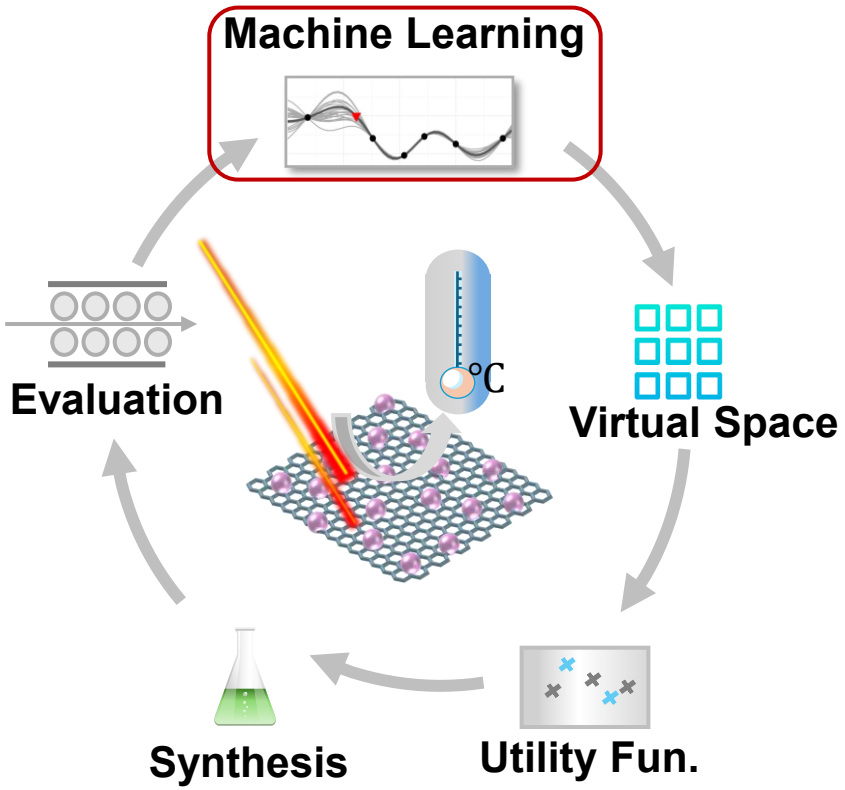
- In this study, τ **decreases over time**:
  - $\tau = \frac{1}{2(T-1)}$
- T: Iteration index (1, 2, 3, ...)
- Goal: **Shift emphasis** from exploration to exploitation as the search progresses





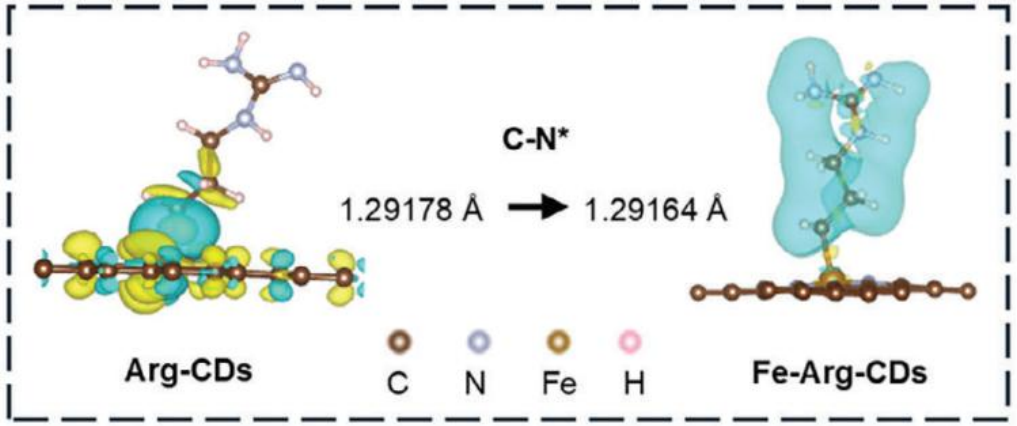
After just three iterations, a novel 105 low-silver lead-free solder is identified, demonstrating exceptional strength and ductility

Application Cases :



Li, T., Cao, B., Su, T., Lin, L., Wang, D., Liu, X., ... & Zhang, T. Y. (2025). Small, 21(5), 2408750.

Nanozyme System design

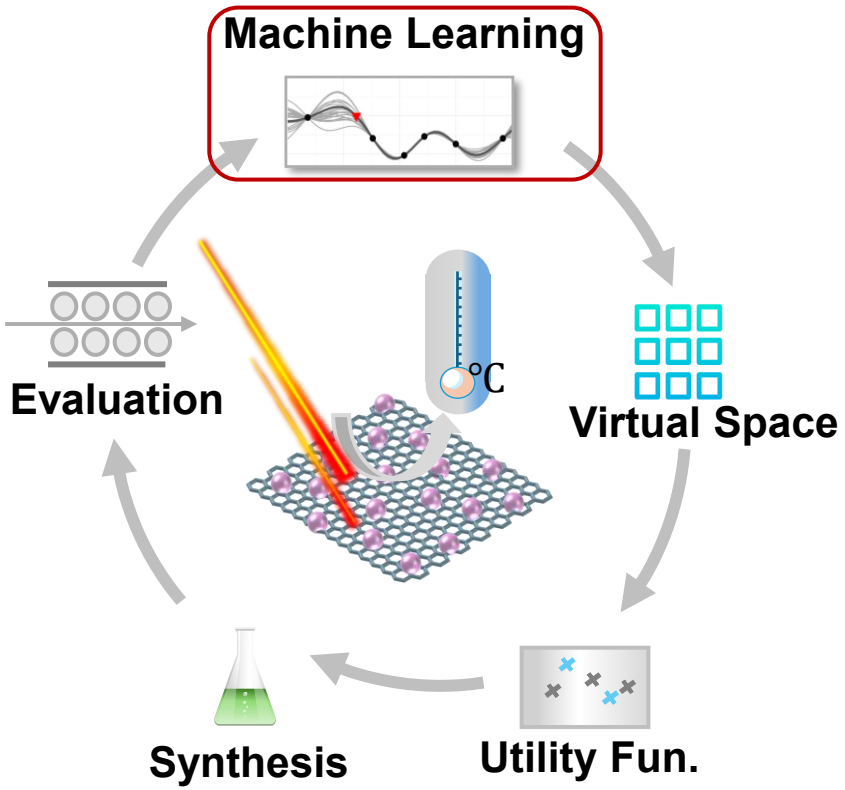


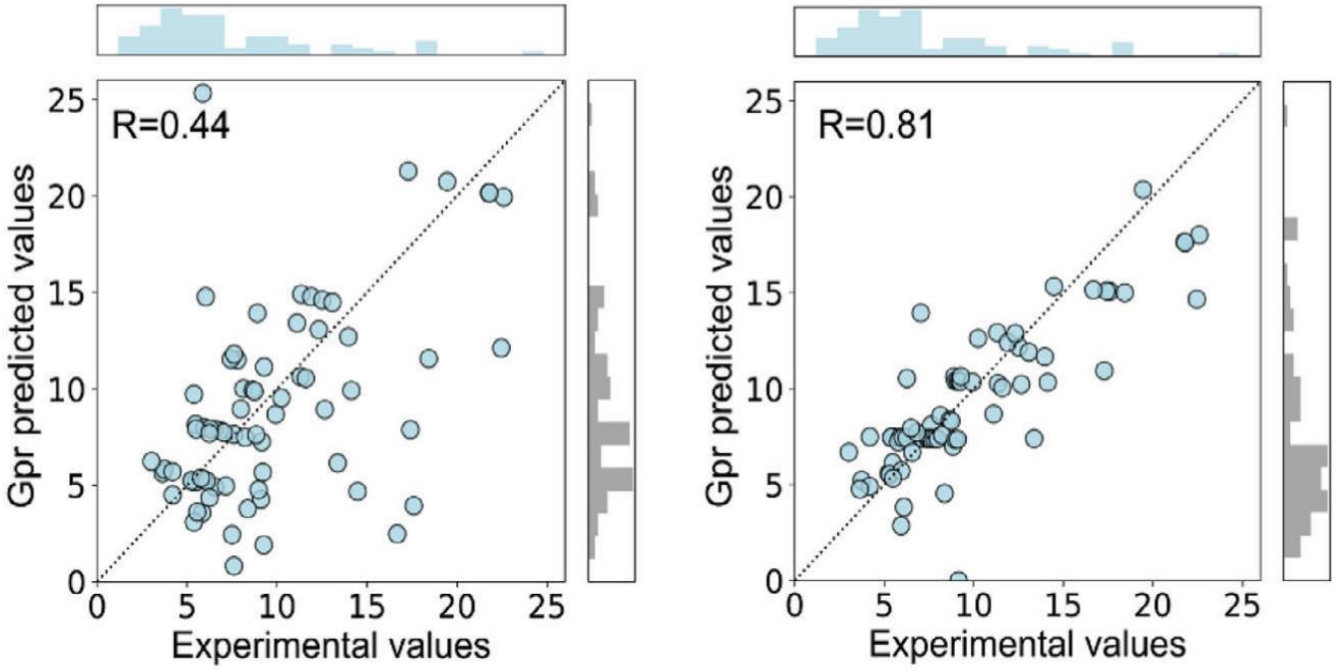
- GPR model uses a **Gaussian radial basis function (RBF) kernel**:

$$K(X_i, X_j) = \exp\left(-\frac{\|X_i - X_j\|^2}{2\vartheta^2}\right)$$

- $\|\cdot\|$: Euclidean distance
- X_i, X_j: Data points (input vectors)
- ϑ: Length scale parameter controlling kernel smoothness

Application Cases :





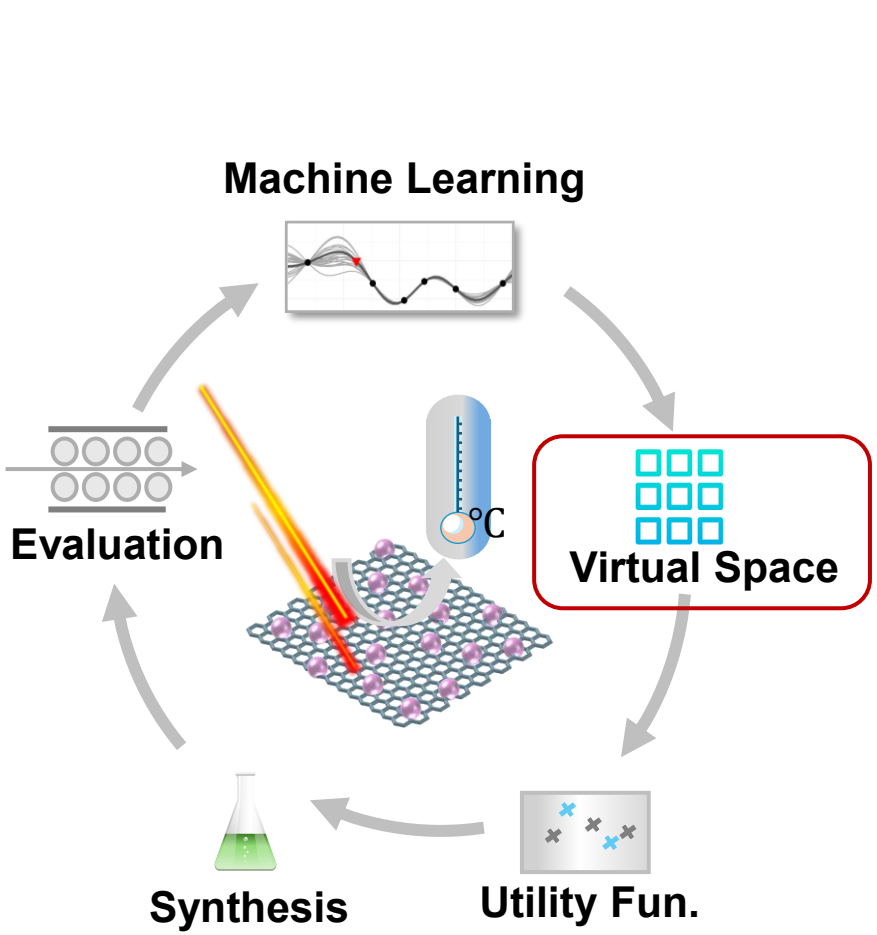
- GGMF measures variation and coherence across length scales:

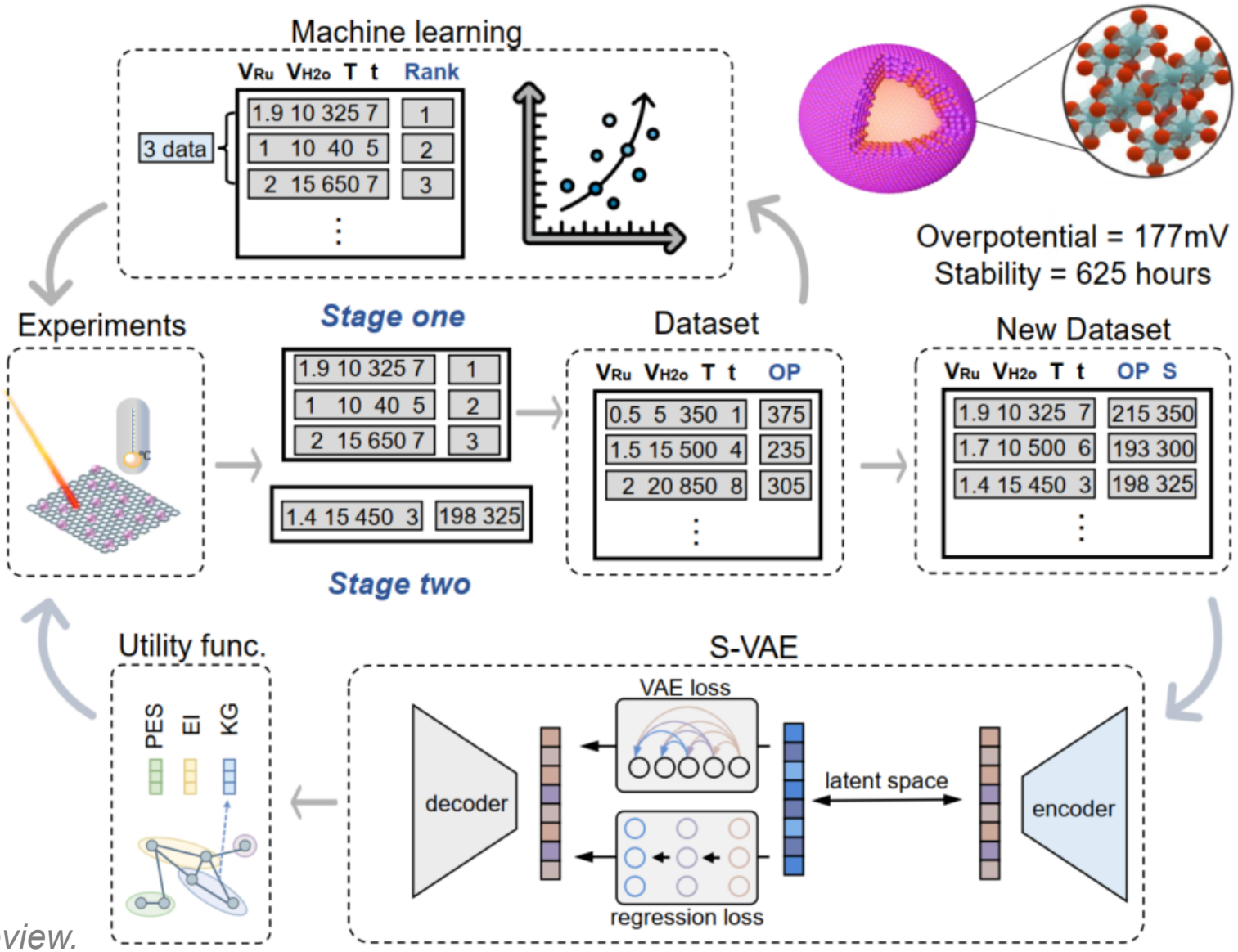
$$\mathrm{GGMF} = \frac{\sqrt{n}\sum_{i=1}^{n}\left(\ln(\vartheta_i+1)-\bar{\vartheta}\right)^2}{\bar{\vartheta}}$$

- $\bar{\vartheta} = \frac{1}{n}\sum_{i=1}^{n}\ln(\vartheta_i+1)$

Li, T., Cao, B., Su, T., Lin, L., Wang, D., Liu, X., ... & Zhang, T. Y. (2025). Small, 21(5), 2408750.

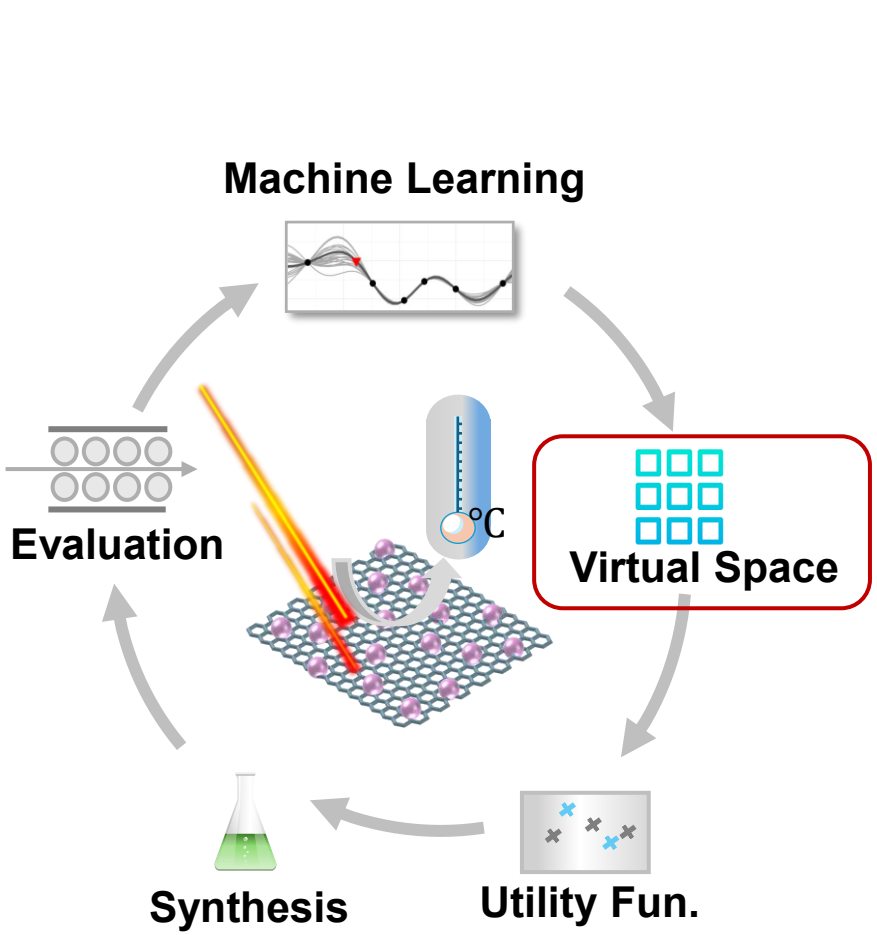
Application Cases :

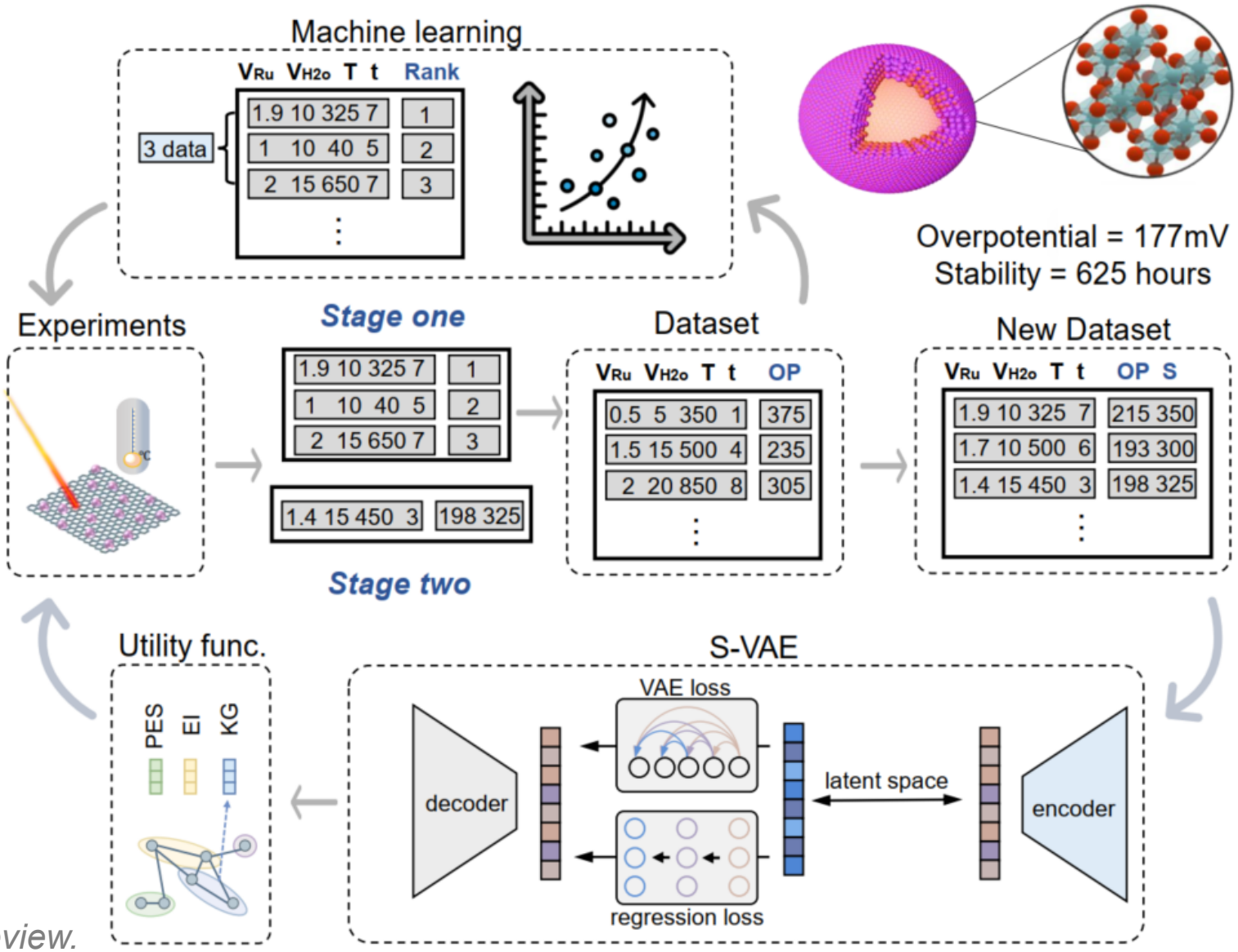




Cao, B., Qin, Y., ... & Zhang, T. Y. (2025). Under review.

Application Cases :





Cao, B., Qin, Y., ... & Zhang, T. Y. (2025). Under review.

Bgolearn

Bgolearn is a computational approach that enables material composition-oriented design and performance-oriented optimization. By leveraging existing experimental data, Bgolearn searches for the optimal material composition design within a specified composition space in order to maximize or minimize the desired performance metric.

Only takes three steps

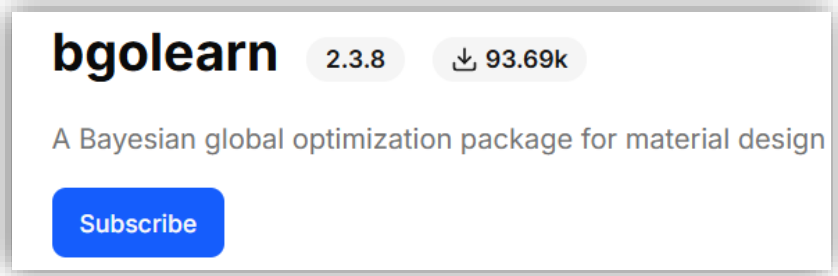
Step 1 :

```
pip install Bgolearn
```

Step 2 :

```
# import your dataset (Samples have been characterized)
data = pd.read_csv('data.csv')
x = data.iloc[:,:-1]
y = data.iloc[:,-1]
# virtual samples which have same feature dimension with x
vs = pd.read_csv('VS.csv')
```

Step 3 :




```
# instantiate class
Bgolearn Bgolearn = BGOS.Bgolearn()
# Pass parameters to the function
Mymodel = Bgolearn.fit(data_matrix = x, Measured_response = y, virtual_samples = vs)
# derive the result by EI
Mymodel.EI()
```

香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Thank you for your attentions!